Studio Nx

S1-12 오디오 회귀 테스트 기반 — 함수 설명 합본 (JS 305~307)

2026-10-21

목차

1. ai-notes/javascript/S1-12 - 오디오 회귀 테스트/305_audio_regression_metrics.md
2. ai-notes/javascript/S1-12 - 오디오 회귀 테스트/306_audio_regression_runner.md
3. ai-notes/javascript/S1-12 - 오디오 회귀 테스트/307_audio_regression_run.md

JavaScript 305 — 비교 지표 (`tests/audio/lib/metrics.mjs`)

항목	내용
파일	`studionx/tests/audio/lib/metrics.mjs`
작업	S1-12 오디오 회귀 테스트 기반: 두 렌더링이 같은 소리인지 판정하는 지표 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 304 장치 화면의 Sampler 연결, 다음 306 실행 도구

1. 하는 일

현재 렌더링과 골든을 세 가지 숫자로 비교하고, 허용 오차 안이면 통과로 판정합니다. 브라우저가 필요 없는 순수 계산이라 Node에서 바로 시험합니다.

지표	계산	기본 허용	잡아내는 것
샘플 최대 차이	모든 샘플에서 두 값의 차이 절댓값의 최댓값	0.001 (-60 dBFS)	한 군데만 크게 달라진 경우 (노트 하나, 클릭 소리)
차이 RMS	두 신호의 차이를 만들어 제곱평균제곱근을 dBFS로	-80 dBFS	전체적으로 조금씩 달라진 경우 (필터, 이펙트 계수)
LUFS 차이	통합 라우드니스(ITU-R BS.1770)의 차이	0.05 LU	사람이 느끼는 음량이 달라진 경우
최소 피크	현재 렌더링의 피크	0.001	소리가 아예 안 나는 경우

```
compareRenders(현재, 골든, 샘플레이트, 허용오차)
  길이가 다르면 —▶ 바로 실패
  피크가 너무 작음 —▶ 실패 (빈 렌더링이 빈 골든과 같다고 통과하는 일을 막음)
  diffMetrics —▶ 최대 차이, 차이 RMS
  integratedLufs —▶ 현재와 골든의 LUFS, 그 차이
  하나라도 허용 오차를 넘으면 failures 에 이유를 적고 pass = false
```

2. 언제 쓰나

쓰는 곳	하는 일
`run.mjs` (307)	프로젝트마다 현재 렌더링과 골든을 비교합니다.
`selftest.mjs` (307)	지표가 맞는지, 일부러 바꾼 소리를 잡는지 확인합니다.

3. 코드

3.1 단순 계산

```
export function toDb(x) {
  return x > 0 ? 20 * Math.log10(x) : -Infinity;
}
```

```
export function peakOf(channels) {
  let p = 0;
  for (const ch of channels) for (let i = 0; i < ch.length; i++) { const v = Math.abs(ch[i]); if (v > p)
p = v; }
  return p;
}
```

```
export function diffMetrics(a, b) {
  let maxDiff = 0, sumSq = 0, n = 0;
  for (let c = 0; c < a.length; c++) {
    const x = a[c], y = b[c];
    for (let i = 0; i < x.length; i++) {
      const d = x[i] - y[i];
      const ad = Math.abs(d);
      if (ad > maxDiff) maxDiff = ad;
      sumSq += d * d;
      n += 1;
    }
  }
  return { maxDiff: maxDiff, rmsDiffDb: toDb(Math.sqrt(sumSq / Math.max(1, n))) };
}
```

3.2 통합 라우드니스

```
function kWeightingStages(fs) {
  // 1단계: 높은 음을 올리는 셸빙 필터 (머리 주변의 음향 효과)
  const f1 = 1681.974450955533, g1 = 3.999843853973347, q1 = 0.7071752369554196;
  let k = Math.tan(Math.PI * f1 / fs);
  const vh = Math.pow(10, g1 / 20);
  const vb = Math.pow(vh, 0.4996667741545416);
  let a0 = 1 + k / q1 + k * k;
  const s1 = {
    b: [(vh + vb * k / q1 + k * k) / a0, 2 * (k * k - vh) / a0, (vh - vb * k / q1 + k * k) / a0],
    a: [2 * (k * k - 1) / a0, (1 - k / q1 + k * k) / a0],
  };
  // 2단계: 낮은 음을 깎는 하이패스 필터
  const f2 = 38.13547087602444, q2 = 0.5003270373238773;
  k = Math.tan(Math.PI * f2 / fs);
  a0 = 1 + k / q2 + k * k;
  const s2 = { b: [1 / a0, -2 / a0, 1 / a0], a: [2 * (k * k - 1) / a0, (1 - k / q2 + k * k) / a0] };
  return [s1, s2];
}
```

```
function biquad(x, st) {
  const y = new Float64Array(x.length);
  let z1 = 0, z2 = 0;
  const b0 = st.b[0], b1 = st.b[1], b2 = st.b[2], a1 = st.a[0], a2 = st.a[1];
  for (let i = 0; i < x.length; i++) {
    const v = x[i];
    const o = b0 * v + z1;
    z1 = b1 * v - a1 * o + z2;
    z2 = b2 * v - a2 * o;
    y[i] = o;
  }
```

```
  return y;
}
```

```
export function integratedLufs(channels, sampleRate) {
  const stages = kWeightingStages(sampleRate);
  const filtered = channels.map(function (ch) {
    let x = ch;
    stages.forEach(function (st) { x = biquad(x, st); });
    return x;
  });
  const len = filtered[0].length;
  const block = Math.round(0.4 * sampleRate);
  const step = Math.round(0.1 * sampleRate);
  const blocks = [];
  for (let start = 0; start + block <= len; start += step) {
    let z = 0;
    for (const ch of filtered) {
      let s = 0;
      for (let i = start; i < start + block; i++) s += ch[i] * ch[i];
      z += s / block;
    }
    blocks.push(z);
  }
  const lufsOf = function (z) { return -0.691 + 10 * Math.log10(z); };
  const abs = blocks.filter(function (z) { return z > 0 && lufsOf(z) > -70; });
  if (!abs.length) return -Infinity;
  const meanAbs = abs.reduce(function (s, z) { return s + z; }, 0) / abs.length;
  const gate = lufsOf(meanAbs) - 10;
  const rel = abs.filter(function (z) { return lufsOf(z) > gate; });
  if (!rel.length) return -Infinity;
  return lufsOf(rel.reduce(function (s, z) { return s + z; }, 0) / rel.length);
}
```

3.3 판정

```
export function compareRenders(current, golden, sampleRate, tolerance) {
  const tol = Object.assign({}, DEFAULT_TOLERANCE, tolerance || {});
  const failures = [];
  const peak = peakOf(current);
  if (current.length !== golden.length || current[0].length !== golden[0].length) {
    failures.push('길이가 다름: 현재 ' + current[0].length + ' 프레임, 골든 ' + golden[0].length + ' 프레
임');
    return { pass: false, failures: failures, maxDiff: NaN, rmsDiffDb: NaN, lufsCurrent: NaN, lufsGolde
n: NaN, lufsDiff: NaN, peak: peak };
  }
  if (peak < tol.minPeak) failures.push('소리가 거의 없음: 피크 ' + peak.toExponential(2));
  const d = diffMetrics(current, golden);
  const lc = integratedLufs(current, sampleRate);
  const lg = integratedLufs(golden, sampleRate);
  const lufsDiff = lc === lg ? 0 : Math.abs(lc - lg);
  if (d.maxDiff > tol.maxDiff) failures.push('샘플 최대 차이 ' + d.maxDiff.toExponential(2) + ' > ' + to
l.maxDiff);
  if (d.rmsDiffDb > tol.rmsDb) failures.push('차이 RMS ' + d.rmsDiffDb.toFixed(1) + ' dBFS > ' + tol.rmsD
b + ' dBFS');
  if (!(lufsDiff <= tol.lufs)) failures.push('LUFS 차이 ' + lufsDiff.toFixed(3) + ' LU > ' + tol.lufs + '
```

```
LU');
  return { pass: failures.length === 0, failures: failures, maxDiff: d.maxDiff, rmsDiffDb: d.rmsDiffDb,
lufsCurrent: lc, lufsGolden: lg, lufsDiff: lufsDiff, peak: peak };
}
```

4. 통합 라우드니스 계산

LUFS는 "사람 귀에 얼마나 크게 들리는가"를 숫자로 만든 값입니다. 단순히 샘플 값의 평균을 내면 낮은 소리와 높은 소리를 같은 비중으로 세지만, 귀는 중고음에 더 민감합니다. 그래서 먼저 K 가중 필터를 거칩니다.

단계	내용
K 가중	1단계는 높은 음을 올리는 셸빙 필터(약 +4dB, 1.7kHz 근처), 2단계는 38Hz 아래를 깎는 하이패스 필터입니다. 계수는 샘플레이트에 맞춰 식으로 계산하므로 44.1kHz와 48kHz에서 모두 맞습니다.
블록	0.4초 길이로 0.1초씩 밀면서(75% 겹침) 블록마다 두 채널의 제곱 평균을 더합니다.
절대 문턱	-70 LUFS보다 작은 블록은 버립니다. 무음 구간이 값을 끌어내리지 못하게 합니다.
상대 문턱	남은 블록의 평균에서 10 LU 낮은 값보다 작은 블록을 또 버립니다. 아주 작은 구간(페이드 아웃)이 값을 끌어내리지 못하게 합니다.
결과	남은 블록의 평균을 `-0.691 + 10 log10(평균)`으로 바꿉니다.

997Hz, 진폭 1(풀스케일) 사인을 양쪽 채널에 똑같이 넣으면 약 0 LUFS가 나옵니다. 이 값으로 구현이 맞는지 확인합니다(자체 시험).

5. 최신 문법으로 쓰면

```
// peakOf 와 diffMetrics 의 최대 차이를 최신 문법으로 쓴 모양
export const peakOf = (channels) => Math.max(0, ...channels.map((ch) => ch.reduce((m, v) => Math.max(m,
Math.abs(v)), 0)));

const maxDiff = Math.max(...a.map((x, c) => x.reduce((m, v, i) => Math.max(m, Math.abs(v - b[c][i])),
0)));

// 허용 오차 병합
const tol = { ...DEFAULT_TOLERANCE, ...tolerance };
```

전통 문법 (실제 코드)	최신 문법
`for` 문으로 최댓값 찾기	`reduce`와 `Math.max`
`Object.assign({}, A, B)`	객체 전개 `{ ...A, ...B }`
`function (z) { return ... }`	화살표 함수

여기서 `Math.max(...배열)`은 샘플이 수십만 개이면 호출 스택을 넘길 수 있어서 실제 코드는 `for` 문을 씁니다.

6. 왜 이렇게

선택	이유
지표를 셋으로 나눴습니다	한 가지만 쓰면 놓치는 변화가 있습니다. RMS만 보면 한 군데의 클릭 소리를, 최대 차이만 보면 전체에 퍼진 미세한 변화를, 둘 다 사람이 느끼는 음량 변화는 LUFS 없이 판단하기 어렵습니다. PRD §7.1이 정한 세 가지입니다.

선택	이유
LUFS 허용을 0.05 LU로 아주 좁게 잡았습니다	같은 코드는 같은 소리를 내므로(차이가 1e-7 수준) 좁게 잡아도 오탐이 없고, 믹서 값이 0.1dB만 어긋나도 잡을 수 있습니다. 실험으로 마스터 볼륨 -0.1dB 변경을 실제로 잡는 것을 확인했습니다.
소리가 없으면 별도로 실패시킵니다	렌더링이 통째로 실패해 무음이 나오고 골든도 무음이면 차이가 0이라 통과해 버립니다. 이 사고가 가장 위험하므로 최소 피크를 따로 둡니다.
길이가 다르면 지표를 계산하지 않고 바로 실패입니다	길이가 다르면 샘플을 맞춰 비교할 수 없고, 길이가 바뀐 것 자체가 큰 변화(템포 계산, 꼬리 길이)이기 때문입니다.
LUFS를 직접 구현했습니다	외부 라이브러리 없이 100줄 안팎이고, 샘플레이트별 계수 계산과 두 문턱의 동작을 자체 시험으로 확인할 수 있습니다.
허용 오차를 덮어쓸 수 있게 했습니다	이후 플러그인(WASM 비교 등)은 허용 오차가 더 넓어야 할 수 있습니다. 프로젝트 JSON의 `tolerance`로 프로젝트마다 정합니다.

7. 시험

`studionx/tests/audio/selftest.mjs`의 첫 부분이 브라우저 없이 확인합니다.

- `toDb`: 1은 0dB, 0.1은 -20dB, 0은 `-Infinity`입니다. `peakOf`는 모든 채널의 절댓값 최댓값입니다.
- LUFS: 44.1kHz와 48kHz 모두 997Hz 풀스케일 사인은 약 0 LUFS이고, 진폭 0.1은 정확히 20 LU 낮습니다.
- 문턱: 뒤쪽 무음이나 -40dB 구간의 길이가 2초든 10초든 값이 같고, 무음을 붙여도 0.5 LU 넘게 내려가지 않습니다. 완전한 무음과 400ms보다 짧은 소리는 `-Infinity`입니다.
- `diffMetrics`: 모든 샘플이 0.001 달라지면 최대 0.001, RMS -60dBFS입니다. 같은 신호는 최대 0, RMS `-Infinity`입니다.
- `compareRenders`: 같으면 통과, 0.99배(0.087dB)만 작아도 LUFS로 실패, 1e-6 잡음은 통과, 둘 다 무음이면 실패, 길이가 다르면 실패, 허용 오차를 덮어쓸 수 있습니다.
- 시험 중 발견한 일: 처음에는 "무음을 붙여도 값이 같다"고 기대했는데, 소리와 맞닿은 경계 블록(일부만 소리가 든 블록)이 값에 들어가서 약 0.3 LU 달랐습니다. 표준 알고리즘이 원래 그렇게 동작하고 구현은 맞으므로, 기대값을 "붙인 구간의 길이가 달라도 값이 같다"로 고쳤습니다.

JavaScript 306 — 실행 도구 (`runner.mjs`, `plugins.mjs`, `wavio.mjs`)

항목	내용
파일	`studionx/tests/audio/lib/runner.mjs`, `studionx/tests/audio/lib/plugins.mjs`, `studionx/tests/audio/lib/wavio.mjs`
작업	S1-12 오디오 회귀 테스트 기반: 테스트 프로젝트를 앱의 `renderOffline`으로 렌더링해 오는 도구 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 305 비교 지표, 다음 307 실행 명령과 자체 시험

1. 하는 일

서버(FastAPI)와 DB 없이, 앱의 소리 코드를 그대로 헤드리스 Chrome에서 돌려 테스트 프로젝트의 소리를 얻어옵니다.

```
Node (run.mjs)                          헤드리스 Chrome
  startServer: frontend 폴더를 내줌  ─▶  /__harness.html 을 엽니다
  buildArgs: 프로젝트 JSON → tick 단위의 인자
  render(인자) ──────────────────────▶  시험용 소리를 수식으로 만들고 renderOffline(앱의 코드)을 부름
  채널별 Float32Array 로 받음 ◀──────  결과를 base64 로 돌려줌
  compareRenders(305)로 골든과 비교
```

파일	맡은 일
`runner.mjs`	정적 서버, Chrome 실행과 DevTools 연결, 렌더링 호출, 프로젝트 JSON 변환
`plugins.mjs`	플러그인 정의(`param_schema`)를 DB 시드 SQL에서 읽기
`wavio.mjs`	골든 WAV를 32bit float로 쓰고 읽기

2. 언제 쓰나

쓰는 곳	하는 일
`run.mjs` (307)	`openRenderer`로 렌더링기를 열고, 프로젝트마다 `buildArgs`와 `render`를 부릅니다. 골든은 `readWav`와 `writeFloatWav`로 다룹니다.
`selftest.mjs` (307)	같은 도구로 프로젝트를 일부러 바꿔 렌더링해 실패하는지 확인합니다.

3. 코드

3.1 프로젝트 JSON 변환과 골든 경로

```
export function loadProjects(only) {
  return fs.readdirSync(PROJECTS_DIR)
    .filter(function (f) { return f.endsWith('.json'); })
    .sort()
    .map(function (f) { return JSON.parse(fs.readFileSync(path.join(PROJECTS_DIR, f), 'utf-8')); })
```

```
    .filter(function (p) { return !only || only.indexOf(p.name) >= 0; });
}
```

```
export function buildArgs(p, schemas) {
  const sch = schemas || loadPluginSchemas();
  const need = function (code) {
    if (!sch[code]) throw new Error('프로젝트 ' + p.name + ': 알 수 없는 플러그인 ' + code);
    return sch[code];
  };
  const tracks = (p.tracks || []).map(function (t) {
    return { track_id: t.id, track_type: t.type, volume_db: t.volume_db || 0, pan: t.pan || 0, is_muted:
!!t.mute, is_solo: !!t.solo };
  });
  if (!tracks.some(function (t) { return t.track_type === 'MASTER'; })) tracks.push({ track_id: 999, tra
ck_type: 'MASTER', volume_db: p.master_db || 0, pan: 0, is_muted: false, is_solo: false });

  const instruments = {};
  Object.keys(p.instruments || {}).forEach(function (tid) {
    instruments[tid] = p.instruments[tid].map(function (l, i) {
      return { id: Number(tid) * 100 + i + 1, plugin_code: l.plugin, schema: need(l.plugin), params: l.p
arams || {}, volume_db: l.volume_db || 0, pan: l.pan || 0, is_enabled: l.enabled !== false };
    });
  });
  const effects = {};
  Object.keys(p.effects || {}).forEach(function (tid) {
    effects[tid] = p.effects[tid].map(function (e, i) {
      return { id: Number(tid) * 100 + i + 1, plugin_code: e.plugin, schema: need(e.plugin), params: e.p
arams || {}, is_bypassed: !!e.bypass, slot_index: i };
    });
  });
  const audioClips = (p.audioClips || []).map(function (c, i) {
    return {
      clip_id: i + 1, track_id: c.track, sample_id: c.sample,
      start_tick: Math.round(c.startBar * TICKS_PER_BAR), length_tick: Math.round(c.lengthBars * TICKS_P
ER_BAR),
      offset_tick: Math.round((c.offsetBars || 0) * TICKS_PER_BAR), gain_db: c.gain_db || 0, is_looped:
!!c.loop, pitch_semitones: c.pitch || 0,
    };
  });
  const midiClips = (p.midiClips || []).map(function (c) {
    return {
      track_id: c.track, start_tick: Math.round(c.startBar * TICKS_PER_BAR), length_tick: Math.round(c.l
engthBars * TICKS_PER_BAR), offset_tick: 0,
      notes: c.notes.map(function (n) {
        return { pitch: n.pitch, velocity: n.velocity == null ? 100 : n.velocity, start_tick: Math.round
(n.startBeat * TICKS_PER_BEAT), length_tick: Math.round(n.lengthBeat * TICKS_PER_BEAT) };
      }),
    };
  });
  return {
    samples: p.samples || {}, tracks: tracks, instruments: instruments, effects: effects, audioClips: au
dioClips, midiClips: midiClips,
    bpm: p.bpm || 120,
    tempoChanges: (p.tempoChanges || []).map(function (c) { return { tick: Math.round(c.bar * TICKS_PER_
BAR), bpm: c.bpm, ramp: c.ramp || 'STEP' }; }),
    startTick: Math.round((p.startBar || 0) * TICKS_PER_BAR),
```

```
    endTick: Math.round(p.endBar * TICKS_PER_BAR), tailSec: p.tailSec || 0, sampleRate: p.sampleRate ||
44100,
  };
}
```

```
export function goldenPath(name) {
  return path.join(GOLDEN_DIR, name + '.wav');
}
```

3.2 서버, 브라우저, 렌더링

```
export function startServer() {
  return new Promise(function (resolve) {
    const server = http.createServer(function (req, res) {
      const url = decodeURIComponent((req.url || '/').split('?')[0]);
      if (url === '/__harness.html') {
        res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
        res.end(HARNESS);
        return;
      }
      const file = path.join(FRONTEND, url);
      if (file.indexOf(FRONTEND) !== 0 || !fs.existsSync(file) || fs.statSync(file).isDirectory()) {
        res.writeHead(404);
        res.end('not found');
        return;
      }
      res.writeHead(200, { 'Content-Type': MIME[path.extname(file)] || 'application/octet-stream' });
      res.end(fs.readFileSync(file));
    });
    server.listen(0, '127.0.0.1', function () { resolve({ server: server, port: server.address().port
}); });
  });
}
```

```
async function launchChrome() {
  const exe = process.env.CHROME_PATH || 'C:/Program Files/Google/Chrome/Application/chrome.exe';
  if (!fs.existsSync(exe)) throw new Error('Chrome 을 찾지 못했습니다: ' + exe + ' (환경 변수 CHROME_PATH
로 지정할 수 있습니다)');
  const port = 9600 + Math.floor(Math.random() * 300);
  const dir = path.join(os.tmpdir(), 'studionx_audio_regression_' + port);
  fs.rmSync(dir, { recursive: true, force: true });
  const chrome = spawn(exe, ['--headless=new', '--remote-debugging-port=' + port, '--user-data-dir=' + d
ir, '--autoplay-policy=no-user-gesture-required', 'about:blank'], { stdio: 'ignore' });
  let target = null;
  for (let i = 0; i < 40 && !target; i++) {
    await new Promise(function (r) { setTimeout(r, 250); });
    try {
      const list = await (await fetch('http://127.0.0.1:' + port + '/json')).json();
      target = list.filter(function (x) { return x.type === 'page'; })[0] || null;
    } catch (e) { /* 아직 안 떴음 */ }
  }
  if (!target) { chrome.kill(); throw new Error('Chrome 에 연결하지 못했습니다'); }
  const ws = new WebSocket(target.webSocketDebuggerUrl);
  await new Promise(function (r) { ws.addEventListener('open', r); });
  let id = 0;
```

```
  const pending = new Map();
  ws.addEventListener('message', function (e) {
    const m = JSON.parse(e.data);
    if (m.id && pending.has(m.id)) pending.get(m.id)(m);
  });
  const send = function (method, params) {
    return new Promise(function (r) { const i = ++id; pending.set(i, r); ws.send(JSON.stringify({ id: i,
method: method, params: params || {} })); });
  };
  await send('Page.enable');
  await send('Runtime.enable');
  const version = (await send('Browser.getVersion')).result.product;
  return {
    send: send,
    version: version,
    close: function () {
      try { ws.close(); } catch (e) { /* 무시 */ }
      chrome.kill();
      // Chrome 이 아직 파일을 잡고 있을 수 있어서, 임시 폴더 지우기는 실패해도 넘어갑니다 (다음 실행이 지
우고 시작)
      try { fs.rmSync(dir, { recursive: true, force: true }); } catch (e) { /* 무시 */ }
    },
  };
}
```

```
export async function openRenderer() {
  const srv = await startServer();
  const browser = await launchChrome();
  await browser.send('Page.navigate', { url: 'http://127.0.0.1:' + srv.port + '/__harness.html' });
  for (let i = 0; i < 80; i++) {
    const r = await browser.send('Runtime.evaluate', { expression: 'window.__ready === true', returnByVa
lue: true });
    if (r.result && r.result.result && r.result.result.value === true) break;
    await new Promise(function (res) { setTimeout(res, 150); });
    if (i === 79) { browser.close(); srv.server.close(); throw new Error('하니스 페이지가 준비되지 않았습
니다'); }
  }
  return {
    version: browser.version,
    render: async function (args) {
      const r = await browser.send('Runtime.evaluate', {
        expression: 'window.__render(' + JSON.stringify(args) + ')',
        awaitPromise: true, returnByValue: true,
      });
      if (r.result && r.result.exceptionDetails) {
        const d = r.result.exceptionDetails;
        throw new Error('렌더링 실패: ' + (d.exception && d.exception.description ? d.exception.descripti
on : d.text));
      }
      const v = r.result.result.value;
      const channels = v.channels.map(function (b64) {
        const buf = Buffer.from(b64, 'base64');
        return new Float32Array(buf.buffer.slice(buf.byteOffset, buf.byteOffset + buf.byteLength));
      });
      return { channels: channels, sampleRate: v.sampleRate, frames: v.frames };
    },
```

```
    close: function () { browser.close(); srv.server.close(); },
  };
}
```

3.3 플러그인 정의

```
export function loadPluginSchemas() {
  const sql = fs.readFileSync(SEED, 'utf-8');
  const out = {};
  const re = /\(\s*'(\w+)'\s*,\s*'[^']*'\s*,\s*'(?:INSTRUMENT|EFFECT|MIDI_EFFECT)'\s*,\s*'\w+'\s*,\s
*'[^']*'\s*,\s*'(\[\{.*?\}\])'\s*,/g;
  let m;
  while ((m = re.exec(sql)) !== null) out[m[1]] = JSON.parse(m[2]);
  return out;
}
```

3.4 WAV 읽고 쓰기

```
export function writeFloatWav(file, channels, sampleRate) {
  const bytes = encodeWav(channels, sampleRate, 32, { gain: 1, dither: false });
  fs.writeFileSync(file, bytes);
  return bytes.length;
}
```

```
export function readWav(file) {
  const buf = fs.readFileSync(file);
  const v = new DataView(buf.buffer, buf.byteOffset, buf.byteLength);
  const tag = function (o) { return String.fromCharCode(buf[o], buf[o + 1], buf[o + 2], buf[o + 3]); };
  if (tag(0) !== 'RIFF' || tag(8) !== 'WAVE') throw new Error('WAV 파일이 아닙니다: ' + file);
  let pos = 12, fmt = null, dataOff = -1, dataLen = 0;
  while (pos + 8 <= buf.length) {
    const id = tag(pos), size = v.getUint32(pos + 4, true);
    if (id === 'fmt ') fmt = { format: v.getUint16(pos + 8, true), ch: v.getUint16(pos + 10, true), rat
e: v.getUint32(pos + 12, true), bits: v.getUint16(pos + 22, true) };
    if (id === 'data') { dataOff = pos + 8; dataLen = size; break; }
    pos += 8 + size + (size % 2);
  }
  if (!fmt || dataOff < 0) throw new Error('fmt 나 data 조각이 없습니다: ' + file);
  const bytes = fmt.bits / 8;
  const frames = Math.floor(dataLen / (bytes * fmt.ch));
  const channels = [];
  for (let c = 0; c < fmt.ch; c++) channels.push(new Float32Array(frames));
  for (let i = 0; i < frames; i++) {
    for (let c = 0; c < fmt.ch; c++) {
      const o = dataOff + (i * fmt.ch + c) * bytes;
      let x;
      if (fmt.format === 3 && fmt.bits === 32) x = v.getFloat32(o, true);
      else if (fmt.bits === 16) x = v.getInt16(o, true) / 32768;
      else if (fmt.bits === 24) { let n = buf[o] | (buf[o + 1] << 8) | (buf[o + 2] << 16); if (n & 0x800
000) n -= 0x1000000; x = n / 8388608; }
      else throw new Error('지원하지 않는 형식: ' + fmt.format + ' / ' + fmt.bits + 'bit');
      channels[c][i] = x;
    }
  }
```

```
  return { channels: channels, sampleRate: fmt.rate, bitDepth: fmt.bits, frames: frames };
}
```

4. 프로젝트 JSON

사람이 읽고 고치기 쉽게 마디와 박 단위로 씁니다. `buildArgs`가 앱이 받는 tick 단위(1박 960, 1마디 3840)로 바꿉니다.

항목	내용
`bpm`, `tempoChanges`	시작 템포와 템포 변경(`bar`, `bpm`, `ramp`)
`sampleRate`, `startBar`, `endBar`, `tailSec`	렌더링 범위와 꼬리
`samples`	시험용 소리를 만드는 식. 종류는 `sine`, `saw`, `kick`, `noise`, `pluck`이고 `freq`, `secs`, `amp`, `decay`, `seed`를 줍니다
`tracks`	`id`, `type`(`MIDI`, `AUDIO`), `volume_db`, `pan`, `mute`, `solo`. MASTER는 자동으로 붙고 `master_db`로 음량을 줍니다
`instruments`, `effects`	트랙 번호별 목록. `plugin`(코드), `params`(바꿀 값만), 악기 층의 `volume_db`, `pan`
`midiClips`, `audioClips`	클립의 위치(`startBar`, `lengthBars`), MIDI 노트(`startBeat`, `lengthBeat`, `pitch`, `velocity`), 오디오 클립의 `gain_db`, `loop`, `pitch`, `offsetBars`
`tolerance`	허용 오차를 프로젝트마다 덮어씀 (생략하면 기본값)

`params`에는 바꿀 값만 적습니다. 나머지는 DB 시드의 기본값이 채웁니다. 그래서 시드의 기본값이 바뀌면 이 테스트가 그 변화를 잡습니다.

5. 최신 문법으로 쓰면

```
// loadProjects 와 소리 생성 부분을 최신 문법으로 쓴 모양
export const loadProjects = (only) =>
  fs.readdirSync(PROJECTS_DIR)
    .filter((f) => f.endsWith('.json'))
    .sort()
    .map((f) => JSON.parse(fs.readFileSync(path.join(PROJECTS_DIR, f), 'utf-8')))
    .filter((p) => !only || only.includes(p.name));

const tracks = (p.tracks ?? []).map(({ id, type, volume_db = 0, pan = 0, mute, solo }) =>
  ({ track_id: id, track_type: type, volume_db, pan, is_muted: !!mute, is_solo: !!solo }));

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
```

전통 문법 (실제 코드)	최신 문법
`only.indexOf(p.name) >= 0`	`only.includes(p.name)`
`t.volume_db || 0`	구조 분해의 기본값 `volume_db = 0`
`(p.tracks || [])`	`p.tracks ?? []`
`function (r) { setTimeout(r, 250) }` 을 감싼 `Promise`	화살표 함수 `(r) => setTimeout(r, 250)`

6. 왜 이렇게

선택	이유
앱의 `renderOffline`을 그대로 부릅니다	테스트가 따로 만든 소리 계산이 아니라 사용자가 듣고 내보내는 소리와 같은 코드를 검사해야 합니다. 앱 코드가 바뀌면 테스트가 바로 그 변화를 봅니다.
서버와 DB 없이 돕니다	정적 서버만 띄워 `frontend`를 내주고, 플러그인 정의는 시드 SQL에서 읽습니다. 로그인, DB, 네트워크 문제로 테스트가 흔들리지 않고, 어느 컴퓨터에서든 Chrome만 있으면 됩니다.
플러그인 정의를 시드에서 읽습니다	실제 서버가 주는 `param_schema`와 같은 값입니다. 시드의 기본값이나 범위가 바뀌면 소리가 달라지는데, 이것도 회귀로 잡히게 합니다.
시험용 소리를 수식으로 만듭니다	바이너리 샘플 파일을 저장소에 넣지 않아도 되고, 소리의 의미(440Hz 사인, 킥, 노이즈)가 JSON에 그대로 보입니다. 노이즈는 시드를 준 선형 합동 생성기라 늘 같은 값입니다.
골든을 32bit float WAV로 저장합니다	렌더링 결과를 손실 없이 담습니다. 16bit로 줄이면 양자화 잡음이 비교 지표에 섞입니다. 앱이 내보내는 `encodeWav`를 그대로 써서, 파일 형식도 앱의 출력과 같습니다.
프로젝트를 5개로, 길이를 2마디(약 4초)로 했습니다	골든 5개가 약 8MB입니다. 길게 만들면 저장소가 빠르게 커지고, 짧아도 악기, 이펙트, 믹서, 템포 변경의 경로를 모두 지나갑니다.
임시 폴더 삭제 실패는 넘어갑니다	Windows에서 Chrome을 끈 직후에는 파일이 잠겨 있어 삭제가 거부됩니다(처음 실행에서 `EPERM`이 나서 알게 됨). 임시 폴더는 다음 실행이 지우고 시작하므로 남아도 해가 없습니다.
Chrome 위치는 환경 변수로 바꿀 수 있습니다	컴퓨터마다 설치 위치가 다를 수 있습니다. 기본은 이 프로젝트가 쓰는 경로입니다.

7. 시험

- `selftest.mjs`의 WAV 시험: 32bit float 스테레오를 쓰고 읽으면 한 비트도 틀리지 않고 돌아옵니다(채널 수, 프레임 수, 샘플레이트, 비트 깊이 포함).
- 프로젝트 5개가 모두 변환과 렌더링을 마치고 골든과 통과하며, 샘플레이트와 프레임 수가 골든과 같습니다.
- 재현성: 같은 프로젝트를 다시 렌더링하면 최대 차이가 1e-6 이하입니다. 실제로는 0에서 약 1.8e-7 사이였습니다(32bit float의 반올림 한계).
- 시험 중 발견한 일: 처음 실행에서 렌더링은 끝났는데 Chrome을 닫는 순간 임시 폴더 삭제가 `EPERM`으로 거부되어 프로그램이 오류로 끝났습니다. 렌더링 결과는 이미 받았으므로 삭제 실패를 무시하도록 고쳤습니다.

JavaScript 307 — 실행 명령과 자체 시험 (`run.mjs`, `selftest.mjs`)

항목	내용
파일	`studionx/tests/audio/run.mjs`, `studionx/tests/audio/selftest.mjs`, `studionx/tests/audio/README.md`
작업	S1-12 오디오 회귀 테스트 기반: 비교 실행, 골든 갱신, 테스트 도구 자체 시험 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 306 실행 도구, 다음 없음 (S1-12의 마지막 문서)

1. 하는 일

`run.mjs`가 소리를 건드린 뒤에 돌리는 명령 하나입니다. 모든 테스트 프로젝트를 렌더링해 골든과 비교하고, 하나라도 달라지면 종료 코드 1로 끝납니다. 의도한 변경이면 같은 명령에 `--update`와 이유를 붙여 골든을 새로 씁니다.

명령	동작
`node studionx/tests/audio/run.mjs`	모든 프로젝트를 비교합니다. 표로 결과와 지표를 보여 줍니다.
`... --only synth_basic,sampler_drums`	이름으로 골라서 비교합니다.
`... --update "변경 이유"`	골든을 새로 쓰고, 이전 골든과의 차이와 이유를 `golden/CHANGELOG.md`에 한 줄 남깁니다. 이유가 없으면 거부합니다.
`node studionx/tests/audio/selftest.mjs`	테스트 도구가 믿을 만한지 확인합니다 (62건).

실행 결과의 모양입니다.

```
브라우저: Chrome/155.0.8059.85
프로젝트            결과  최대 차이   RMS dBFS  LUFS 차이  골든 LUFS  시간
effects_chain     PASS  0.000000    -inf      0.000     -6.7       0.2s
synth_basic       FAIL  0.026516    -48.2     0.285     -16.2      0.2s
        - 샘플 최대 차이 2.65e-2 > 0.001
        - 차이 RMS -48.2 dBFS > -80 dBFS
        - LUFS 차이 0.285 LU > 0.05 LU
```

2. 언제 쓰나

상황	할 일
오디오 엔진, 플러그인, 라우팅 코드를 바꿈	`run.mjs`를 돌려 전부 통과하는지 봅니다.
실패함	`out/<이름>.wav`를 들어 보고, 의도한 변화인지 판단합니다.
의도한 변화였음	`--update "이유"`로 골든을 갱신하고 변경 기록과 함께 커밋합니다.
의도하지 않은 변화였음	코드를 고칩니다. 골든은 건드리지 않습니다.
테스트 도구 자체를 고침	`selftest.mjs`를 돌립니다.

3. 코드

3.1 실행 명령 (`run.mjs`)

```
function appendChangelog(rows, reason, chrome) {
  if (!fs.existsSync(CHANGELOG)) {
    fs.writeFileSync(CHANGELOG, [
      '# 골든 WAV 변경 기록',
      '',
      '의도적으로 소리를 바꾼 경우에만 골든을 새로 씁니다 (`node studionx/tests/audio/run.mjs --update "이유"`). 이유와 이전 골든과의 차이를 여기에 남깁니다.',
      '',
      '| 날짜 | 프로젝트 | 이유 | 이전과의 차이 (최대 차이, RMS dBFS, LUFS 차이) | 브라우저 |',
      '|---|---|---|---|---|',
      '',
    ].join('\n'), 'utf-8');
  }
  const date = new Date().toISOString().slice(0, 10);
  const text = rows.map(function (r) {
    const diff = r.cmp ? num(r.cmp.maxDiff, 6) + ', ' + num(r.cmp.rmsDiffDb, 1) + ', ' + num(r.cmp.lufsDiff, 3) : '새로 만듦';
    return '| ' + date + ' | `' + r.name + '` | ' + reason.replace(/\|/g, '/') + ' | ' + diff + ' | ' + chrome + ' |';
  }).join('\n') + '\n';
  fs.appendFileSync(CHANGELOG, text, 'utf-8');
}
```

```
async function main() {
  const onlyArg = argValue('--only');
  const only = onlyArg ? onlyArg.split(',') : null;
  const update = process.argv.indexOf('--update') >= 0;
  const reason = update ? (argValue('--update') || '').trim() : '';
  if (update && (!reason || reason.indexOf('--') === 0)) {
    console.error('--update 에는 변경 이유가 필요합니다. 예: --update "Reverb 감쇠 계산 수정"');
    process.exit(2);
  }
  const projects = loadProjects(only);
  if (!projects.length) { console.error('실행할 프로젝트가 없습니다 (--only 이름을 확인하세요)'); process.exit(2); }

  const r = await openRenderer();
  const results = [];
  try {
    for (const p of projects) {
      const t0 = Date.now();
      const out = await r.render(buildArgs(p));
      const ms = Date.now() - t0;
      const gp = goldenPath(p.name);
      const old = fs.existsSync(gp) ? readWav(gp) : null;
      if (update) {
        const cmp = old && old.frames === out.frames ? compareRenders(out.channels, old.channels, out.sampleRate, p.tolerance) : null;
        const bytes = writeFloatWav(gp, out.channels, out.sampleRate);
        results.push({ name: p.name, ms: ms, cmp: cmp, wrote: bytes, pass: true });
        continue;
```

```
      }
      if (!old) { results.push({ name: p.name, ms: ms, pass: false, note: '골든이 없습니다. --update "이유"로 만드세요' }); continue; }
      const cmp = compareRenders(out.channels, old.channels, out.sampleRate, p.tolerance);
      if (!cmp.pass) {
        fs.mkdirSync(OUT_DIR, { recursive: true });
        writeFloatWav(path.join(OUT_DIR, p.name + '.wav'), out.channels, out.sampleRate);
      }
      results.push({ name: p.name, ms: ms, cmp: cmp, pass: cmp.pass });
    }
  } finally {
    r.close();
  }

  if (update) {
    appendChangelog(results, reason, r.version);
    results.forEach(function (x) { console.log('골든 갱신 ' + x.name + ' (' + (x.wrote / 1048576).toFixed(2) + ' MB)' + (x.cmp ? '  이전과 최대 차이 ' + num(x.cmp.maxDiff, 6) : '  새로 만듦')); });
    console.log('변경 기록: ' + path.relative(process.cwd(), CHANGELOG));
    return;
  }

  console.log('브라우저: ' + r.version);
  console.log('프로젝트'.padEnd(18) + '결과  ' + '최대 차이'.padEnd(12) + 'RMS dBFS'.padEnd(10) + 'LUFS 차이'.padEnd(10) + '골든 LUFS  시간');
  results.forEach(function (x) {
    if (!x.cmp) { console.log(x.name.padEnd(18) + 'FAIL  ' + x.note); return; }
    console.log(x.name.padEnd(18) + (x.pass ? 'PASS  ' : 'FAIL  ') + num(x.cmp.maxDiff, 6).padEnd(12) + num(x.cmp.rmsDiffDb, 1).padEnd(10) + num(x.cmp.lufsDiff, 3).padEnd(10) + num(x.cmp.lufsGolden, 1).padEnd(11) + (x.ms / 1000).toFixed(1) + 's');
    x.cmp.failures.forEach(function (f) { console.log('        - ' + f); });
  });
  const failed = results.filter(function (x) { return !x.pass; });
  console.log('\n' + (results.length - failed.length) + '/' + results.length + ' 통과');
  if (failed.length) {
    console.log('실패한 렌더링은 studionx/tests/audio/out/ 에 저장했습니다. 의도한 변경이면 --update "이유"로 골든을 갱신하세요.');
    process.exit(1);
  }
}
```

4. 자체 시험 (`selftest.mjs`)

회귀 테스트가 거짓으로 통과하면 아무것도 지켜 주지 못하므로, 도구 자체를 세 가지로 확인합니다.

종류	확인하는 것	건수
지표	LUFS가 표준값과 맞는지, 문턱 동작, 차이 계산, 판정 규칙, WAV 왕복 (305)	21
골든	프로젝트마다 골든이 있는지, 통과하는지, 피크가 풀스케일 아래인지, 샘플레이트와 프레임 수가 같은지 (골든 파일 확인 1건과 프로젝트당 3건)	16
재현성	같은 프로젝트를 다시 렌더링해도 최대 차이가 1e-6 이하인지	1
민감도	소리를 조금 바꾼 프로젝트가 골든과 비교해 반드시 실패하는지	22
소리 없음	모든 클립이 빠진 렌더링이 "소리가 거의 없음"으로 실패하는지	1

종류	확인하는 것	건수
프로젝트 수	테스트 프로젝트가 3개에서 5개 사이인지	1

민감도 시험이 가장 중요합니다. 프로젝트를 복사해 한 군데만 일부러 바꾸고, 그 결과가 골든과 비교해 실패해야 통과입니다.

영역	바꾼 것 (모두 실패해야 함)
Synth	컷오프 +5%, 마스터 볼륨 -0.1dB, 노트 하나 반음 올림, LFO 깊이 0.35에서 0.30
Sampler	One-Shot에서 Classic으로, 스네어 Decay, 하이햇 벨로시티 하나, 베이스 Root
이펙트	Delay 시간 0.25에서 0.26, Compressor 임계값 1dB, Reverb 감쇠 0.1초, EQ 0.5dB, 이펙트 순서(Compressor와 Delay), Limiter 천장
믹서	팬 0.1, 뮤트 해제, 클립 gain 1dB, 클립 음정 반음 하나, 클립 시작 위치 0.02마디
템포	템포 변경 140에서 141 BPM, 변경 위치 0.1마디, Synth 컷오프 40Hz

5. 최신 문법으로 쓰면

```
// argValue 와 결과 표 출력을 최신 문법으로 쓴 모양
const argValue = (flag) => {
  const i = process.argv.indexOf(flag);
  return i >= 0 ? process.argv[i + 1] : null;
};

results.forEach(({ name, pass, cmp }) =>
  console.log(`${name.padEnd(18)}${pass ? 'PASS' : 'FAIL'}  ${cmp.maxDiff.toFixed(6)}`));

const changelogRows = rows.map((r) => `| ${date} | \`${r.name}\` | ${reason} |`).join('\n');
```

전통 문법 (실제 코드)	최신 문법
`function (x) { return ... }`	화살표 함수
`'`	' + date + '
`x.cmp ? ... : ...`를 풀어 쓴 `if`	구조 분해 `({ name, pass, cmp })`

6. 왜 이렇게

선택	이유
골든 갱신에 이유를 필수로 합니다	PRD §7.1이 "의도적으로 소리를 바꾼 경우에만, 변경 이유를 기록하고" 갱신하라고 했습니다. 이유 없이 `--update`를 쓰면 실패한 테스트를 통과시키려고 골든을 덮어쓰는 일이 쉬워집니다. 명령이 이유를 요구하면 그 순간 한 번 생각하게 됩니다.
변경 기록에 이전 골든과의 차이와 브라우저 버전을 남깁니다	나중에 "이때 소리가 얼마나 달라졌고 어떤 Chrome이었는지"를 알 수 있습니다. 브라우저를 올리고 소리가 달라졌다면 이 기록이 원인을 좁혀 줍니다.
실패한 렌더링을 `out/`에 WAV로 저장합니다	숫자만으로는 무엇이 달라졌는지 모릅니다. 들어 보면 바로 알 수 있고, 저장소에는 올리지 않도록 `.gitignore`에 넣었습니다.
종료 코드를 실패 1, 사용 오류 2로 나눴습니다	CI나 스크립트에서 "소리가 달라졌다"와 "실행 자체가 잘못됐다"를 구분합니다.

선택	이유
민감도 시험을 둡니다	허용 오차가 너무 넓으면 실제 변화를 놓치는데, 그것은 통과하는 테스트로는 알 수 없습니다. 소리를 일부러 바꿔 보고 실패하는지 확인해야 허용 오차가 적절하다는 증거가 됩니다.
이펙트 순서 시험은 Compressor와 Delay를 바꿉니다	처음에는 Delay와 Reverb를 바꿨는데 결과가 같아서 시험이 실패했습니다. 둘 다 선형이고 시간에 따라 변하지 않는 필터라서 순서를 바꿔도 결과가 같은 것이 수학적으로 맞습니다. 비선형인 Compressor와 바꾸면 순서가 결과에 영향을 줍니다.
프로젝트의 Gain을 키워 Limiter가 일하게 했습니다	처음에는 소리가 천장(-1dB)보다 작아서 Limiter가 아무것도 하지 않았고, 천장을 -3으로 바꿔도 소리가 같았습니다. 회귀 테스트가 Limiter를 지켜 주려면 Limiter가 실제로 작동하는 소리를 넣어야 합니다.
마스터 볼륨을 내려 피크를 풀스케일 아래로 맞췄습니다	부동소수점 렌더링은 1.0을 넘어도 값이 유지되지만, 실제로 내보내면 잘리는 소리를 기준으로 삼으면 안 됩니다. 처음 만든 프로젝트 둘이 1.3에 가까웠습니다.

7. 시험

- 자체 시험 62건이 모두 통과합니다. 지표 21건, 골든 16건, 재현성 1건, 민감도 22건, 소리 없음 1건, 프로젝트 수 1건입니다.
- 실제 코드 변경으로 확인했습니다. `synth.js`의 최고 음량 계수 0.3을 0.31(약 +3%)로 바꾸고 `run.mjs`를 돌리자, Synth를 쓰는 `synth_basic`(LUFS 차이 0.285 LU)과 `full_mix_tempo`(최대 차이 6e-3)만 실패했고 나머지 세 프로젝트는 통과했습니다. 원래대로 되돌린 뒤에는 다시 5개 모두 통과했습니다. 한 곳의 변화가 그 소리를 쓰는 프로젝트에만 잡히는 것을 확인한 것입니다.
- 시험 중 발견한 일은 세 가지이고 모두 시험이나 프로젝트의 문제였습니다. 이펙트 순서 시험의 기대(Delay와 Reverb는 순서가 결과를 바꾼다)가 수학적으로 틀렸고, Limiter가 작동하지 않는 소리였으며, 무음을 붙인 LUFS 비교의 기대가 표준 알고리즘의 경계 블록 동작과 달랐습니다. 앱 코드의 오류는 없었습니다.